

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA0504

COURSE NAME: Database Management Systems

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS		Total Marks: 50	
Q. No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order (Order ID, Cust ID, Cust Name, Prod ID, Prod Name, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient (PID, P Name, Doc ID, Doc Name, Specialization, Ward No, Ward Name). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment (SID, S Name, Course ID, Course Name, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking (Flight No, Pass ID, Pass Name, Seat, Dept City, Arr City, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library (Member ID, Member Name, Book ID, Title, Author, Genre, Borrow Date). Identify MVDs and decompose to 4NF.	BL3 – Apply	5
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5

8	A retail inventory system has Product (PID, P Name, Supplier ID, Supplier Name, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

*I **SANTHOSH E / 192511470** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E.Santhosh

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1) Analyze the case: An online shopping platform stores Order (OrderID, Cust ID, Cust Name, Prod ID, Prod Name, Qty, Price). Identify FDs and normalize to 3NF.

Given Relation

ORDER (OrderID, Cust ID, Cust Name, Prod ID, Prod Name, Qty, Price)

Assume:

- OrderID $\rightarrow$ Order number
- Cust ID $\rightarrow$ Customer ID
- Cust Name $\rightarrow$ Customer name
- Prod ID $\rightarrow$ Product ID
- Prod Name $\rightarrow$ Product name
- Qty $\rightarrow$ Quantity ordered
- Price $\rightarrow$ Product price

Functional Dependencies

- Cust ID $\rightarrow$ Cust Name
- Prod ID $\rightarrow$ Prod Name, Price
- (OrderID, Prod ID) $\rightarrow$ Qty

The candidate key is:

(Order ID, Prod ID)

because one order can contain multiple products.

Identify Dependencies

There is a partial dependency:

OrderID, Prod ID $\rightarrow$ Qty
Prod ID $\rightarrow$ Prod Name, Price

Since Prod Name and Price depend only on Prod ID, they depend on part of the composite key.

There are also dependencies involving customer information:

Cust ID → Cust Name

So we decompose the relation to remove redundancy and achieve 3NF.

3NF Decomposition

1. CUSTOMER

CUSTOMER (Cust ID, Cust Name)

2. PRODUCT

PRODUCT (Prod ID, Prod Name, Price)

3. ORDER

ORDER (OrderID, Cust ID)

4. ORDER_ITEM

ORDER_ITEM (OrderID, Prod ID, Qty)

```
Database changed
mysql> CREATE TABLE Customer (
  ->     CustID INT PRIMARY KEY,
  ->     CustName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Customer
  -> VALUES
  -> (1, 'Rahul'),
  -> (2, 'Priya'),
  -> (3, 'Arun');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE Product (
  ->     ProdID INT PRIMARY KEY,
  ->     ProdName VARCHAR(50),
  ->     Price DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Product
  -> VALUES
  -> (101, 'Laptop', 65000),
  -> (102, 'Mouse', 800),
  -> (103, 'Keyboard', 1500);
Query OK, 3 rows affected (0.01 sec)
```

CREATE TABLE AND INSERT DATA

```
mysql> CREATE TABLE Orders (  
->     OrderID INT PRIMARY KEY,  
->     CustID INT,  
->     FOREIGN KEY (CustID) REFERENCES Customer(CustID)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> INSERT INTO Orders  
-> VALUES  
-> (1001, 1),  
-> (1002, 2),  
-> (1003, 3);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> CREATE TABLE Order_Item (  
->     OrderID INT,  
->     ProdID INT,  
->     Qty INT,  
->     PRIMARY KEY (OrderID, ProdID),  
->     FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),  
->     FOREIGN KEY (ProdID) REFERENCES Product(ProdID)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> INSERT INTO Order_Item  
-> VALUES  
-> (1001, 101, 1),  
-> (1001, 102, 2),  
-> (1002, 103, 1),  
-> (1003, 102, 3);  
Query OK, 4 rows affected (0.01 sec)  
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Customer;  
+-----+-----+  
| CustID | CustName |  
+-----+-----+  
|      1 | Rahul    |  
|      2 | Priya    |  
|      3 | Arun     |  
+-----+-----+  
3 rows in set (0.01 sec)  
  
mysql> SELECT * FROM Product;  
+-----+-----+-----+  
| ProdID | ProdName | Price |  
+-----+-----+-----+  
|     101 | Laptop   | 65000.00 |  
|     102 | Mouse    | 800.00 |  
|     103 | Keyboard | 1500.00 |  
+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> SELECT * FROM Orders;  
+-----+-----+  
| OrderID | CustID |  
+-----+-----+  
|     1001 |      1 |  
|     1002 |      2 |  
|     1003 |      3 |  
+-----+-----+  
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Order_Item;
```

OrderID	ProdID	Qty
1001	101	1
1001	102	2
1002	103	1
1003	102	3

```
4 rows in set (0.00 sec)
```



```
mysql> SELECT
  -> O.OrderID,
  -> C.CustName,
  -> P.ProdName,
  -> OI.Qty,
  -> P.Price
  -> FROM Orders O
  -> JOIN Customer C
  -> ON O.CustID = C.CustID
  -> JOIN Order_Item OI
  -> ON O.OrderID = OI.OrderID
  -> JOIN Product P
  -> ON OI.ProdID = P.ProdID;
```

OrderID	CustName	ProdName	Qty	Price
1001	Rahul	Laptop	1	65000.00
1001	Rahul	Mouse	2	800.00
1002	Priya	Keyboard	1	1500.00
1003	Arun	Mouse	3	800.00

```
4 rows in set (0.01 sec)
```

Justification

The original relation contains redundant customer and product information because the same customer and product details can appear in many orders. Decomposing it into Customer, Product, Orders, and Order _ Item removes partial and transitive dependencies while preserving the relationships. Hence, the resulting schema satisfies 3NF.

2) A hospital maintains Patient (PID, P Name, Doc ID, Doc Name, Specialization, Ward No, Ward Name). Identify anomalies and normalize to BCNF.

Given Relation

PATIENT (PID, P Name, Doc ID, Doc Name, Specialization, Ward No, Ward Name)
Assume the following functional dependencies:

PID → P Name, Doc ID, Ward No

Doc ID $\rightarrow$ Doc Name, Specialization

Ward No $\rightarrow$ Ward Name

Therefore, there are transitive dependencies:

PID $\rightarrow$ Doc ID $\rightarrow$ Doc Name, Specialization

And

PID $\rightarrow$ Ward No $\rightarrow$ Ward Name

Identify Anomalies

1. Update Anomaly

If a doctor's name changes, it must be updated in multiple patient records.

For example, if Doc ID = D01 appears for 20 patients, the doctor's name is repeated 20 times.

2. Insertion Anomaly

A new doctor cannot easily be added unless a patient is assigned to that doctor.

3. Deletion Anomaly

If the last patient assigned to a doctor is deleted, the doctor's information may also be lost.

BCNF Analysis

A relation is in BCNF when every determinant in a non-trivial functional dependency is a super key.

In the original relation:

Doc ID $\rightarrow$ Doc Name, Specialization

But Doc ID is not a super key of the complete Patient relation.

Similarly:

Ward No $\rightarrow$ Ward Name

but Ward No is not a super key of the complete relation.

Therefore:

Patient is not in BCNF

BCNF DECOMPOSITION

Separate the determinants into their own relations.

1. PATIENT

PATIENT (PID, P Name, Doc ID, Ward No)

2. DOCTOR

DOCTOR (Doc ID, Doc Name, Specialization)

3. WARD

WARD (Ward No, Ward Name)

```
mysql> CREATE TABLE Patient (
->   PID INT PRIMARY KEY,
->   PName VARCHAR(50),
->   DocID VARCHAR(10),
->   WardNo INT,
->   FOREIGN KEY (DocID) REFERENCES Doctor(DocID),
->   FOREIGN KEY (WardNo) REFERENCES Ward(WardNo)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Patient
-> VALUES
-> (1, 'Rahul', 'D01', 101),
-> (2, 'Priya', 'D02', 102),
-> (3, 'Arun', 'D01', 101),
-> (4, 'Sneha', 'D03', 103);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Patient;
+----+-----+-----+-----+
| PID | PName | DocID | WardNo |
+----+-----+-----+-----+
| 1   | Rahul | D01   | 101    |
| 2   | Priya | D02   | 102    |
| 3   | Arun  | D01   | 101    |
| 4   | Sneha | D03   | 103    |
+----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE Doctor (
->   DocID VARCHAR(10) PRIMARY KEY,
->   DocName VARCHAR(50),
->   Specialization VARCHAR(50)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Doctor
-> VALUES
-> ('D01', 'Kumar', 'Cardiology'),
-> ('D02', 'Priya', 'Neurology'),
-> ('D03', 'Ravi', 'Orthopedics');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE Ward (
->   WardNo INT PRIMARY KEY,
->   WardName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Ward
-> VALUES
-> (101, 'General Ward'),
-> (102, 'ICU'),
-> (103, 'Emergency');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```



```
mysql> SELECT * FROM Doctor;
```

DocID	DocName	Specialization
D01	Kumar	Cardiology
D02	Priya	Neurology
D03	Ravi	Orthopedics

```
3 rows in set (0.00 sec)
```



```
mysql> SELECT * FROM Ward;
```

WardNo	WardName
101	General Ward
102	ICU
103	Emergency

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT
-> P.PID,
-> P.PName,
-> D.DocID,
-> D.DocName,
-> D.Specialization,
-> W.WardNo,
-> W.WardName
-> FROM Patient P
-> JOIN Doctor D
-> ON P.DocID = D.DocID
-> JOIN Ward W
-> ON P.WardNo = W.WardNo;
```

PID	PName	DocID	DocName	Specialization	WardNo	WardName
1	Rahul	D01	Kumar	Cardiology	101	General Ward
3	Arun	D01	Kumar	Cardiology	101	General Ward
2	Priya	D02	Priya	Neurology	102	ICU
4	Sneha	D03	Ravi	Orthopedics	103	Emergency

```
4 rows in set (0.00 sec)
```

Justification

The original relation has redundancy because doctor and ward information is repeated for multiple patients. Doc ID and Ward No determine non-key attributes but are not superkeys of the original relation, causing BCNF violations. Separating Doctor and Ward into independent relations removes these dependencies and produces a BCNF design.

3) Case study: A university stores Enrollment (SID, S Name, Course ID, Course Name, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

Given Relation

ENROLLMENT (SID, S Name, Course ID, Course Name, Faculty, Grade)

Functional Dependencies

SID $\rightarrow$ S Name
Course ID $\rightarrow$ Course Name, Faculty
(SID, Course ID) $\rightarrow$ Grade

Candidate Key

- A student can enroll in multiple courses, so SID alone is not enough.
- A course can have multiple students, so Course ID alone is not enough.

Therefore:

(S ID, Course ID)

is the candidate key.

Partial Dependencies

We have:

SID $\rightarrow$ S Name
Course ID $\rightarrow$ Course Name, Faculty

Both depend on parts of the composite key, so the relation is not in 2NF.

2NF DECOMPOSITION

Separate the attributes that depend only on part of the key.

STUDENT

STUDENT (SID, S Name)

COURSE

COURSE (Course ID, Course Name, Faculty)

ENROLLMENT

ENROLLMENT (SID, Course ID, Grade)

CREATE TABLE AND INSERT DATA

```
Database changed
mysql> CREATE TABLE Student (
  ->     SID INT PRIMARY KEY,
  ->     SName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student
  -> VALUES
  -> (101, 'Rahul'),
  -> (102, 'Priya'),
  -> (103, 'Arun');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE Course (
  ->     CourseID VARCHAR(10) PRIMARY KEY,
  ->     CourseName VARCHAR(50),
  ->     Faculty VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Course
  -> VALUES
  -> ('C01', 'DBMS', 'Dr. Kumar'),
  -> ('C02', 'Python', 'Dr. Priya'),
  -> ('C03', 'Java', 'Dr. Ravi');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Course;
+-----+-----+-----+
| CourseID | CourseName | Faculty |
+-----+-----+-----+
| C01      | DBMS       | Dr. Kumar |
| C02      | Python     | Dr. Priya |
| C03      | Java       | Dr. Ravi  |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Enrollment;
+-----+-----+-----+
| SID | CourseID | Grade |
+-----+-----+-----+
| 101 | C01      | A     |
| 101 | C02      | B+    |
| 102 | C01      | A+    |
| 102 | C03      | B     |
| 103 | C02      | A     |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE Enrollment (
->     SID INT,
->     CourseID VARCHAR(10),
->     Grade VARCHAR(5),
->     PRIMARY KEY (SID, CourseID),
->     FOREIGN KEY (SID) REFERENCES Student(SID),
->     FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Enrollment
-> VALUES
-> (101, 'C01', 'A'),
-> (101, 'C02', 'B+'),
-> (102, 'C01', 'A+'),
-> (102, 'C03', 'B'),
-> (103, 'C02', 'A');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+
| SID | SName |
+-----+-----+
| 101 | Rahul |
| 102 | Priya |
| 103 | Arun  |
+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT
->     S.SID,
->     S.SName,
->     C.CourseID,
->     C.CourseName,
->     C.Faculty,
->     E.Grade
-> FROM Student S
-> JOIN Enrollment E
->     ON S.SID = E.SID
-> JOIN Course C
->     ON E.CourseID = C.CourseID;
+-----+-----+-----+-----+-----+-----+
| SID | SName | CourseID | CourseName | Faculty | Grade |
+-----+-----+-----+-----+-----+-----+
| 101 | Rahul | C01      | DBMS       | Dr. Kumar | A      |
| 101 | Rahul | C02      | Python     | Dr. Priya | B+     |
| 102 | Priya | C01      | DBMS       | Dr. Kumar | A+     |
| 102 | Priya | C03      | Java       | Dr. Ravi  | B      |
| 103 | Arun  | C02      | Python     | Dr. Priya | A      |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Justification

The decomposition stores student and course details only once, reducing redundancy and avoiding update anomalies. The ENROLLMENT table retains the relationship between students and courses along with their grades, giving a proper 2NF structure.

4) Given the airline booking case: Booking (Flight No, Pass ID, Pass Name, Seat, Dept City, Arr City, Date). Normalize up to BCNF.

Given Relation

BOOKING (Flight No, Pass ID, Pass Name, Seat, Dept City, Arr City, Date)

Assume these functional dependencies:

Pass ID $\rightarrow$ Pass Name

Flight No $\rightarrow$ Dept City, Arr City, Date

(Flight No, Pass ID) $\rightarrow$ Seat

The candidate key is:

(Flight No, Pass ID)

because a passenger can book multiple flights and a flight can have multiple passengers.

Dependencies

We have:

Pass ID $\rightarrow$ Pass Name

and:

Flight No $\rightarrow$ Dept City, Arr City, Date

Both depend on part of the composite key.

Therefore, the original relation is not even in 2NF.

Decompose it into:

PASSENGER

PASSENGER (Pass ID, Pass Name)

FLIGHT

FLIGHT (Flight No, Dept City, Arr City, Date)

BOOKING

BOOKING (Flight No, Pass ID, Seat)

Now each determinant is a key in its respective relation, so the resulting relations satisfy BCNF.

CREATE TABLE AND INSERT

```
mysql> CREATE TABLE Passenger (  
->     PassID INT PRIMARY KEY,  
->     PassName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Passenger  
-> VALUES  
-> (1, 'Rahul'),  
-> (2, 'Priya'),  
-> (3, 'Arun');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> CREATE TABLE Flight (  
->     FlightNo VARCHAR(10) PRIMARY KEY,  
->     DeptCity VARCHAR(50),  
->     ArrCity VARCHAR(50),  
->     FlightDate DATE  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Flight  
-> VALUES  
-> ('F101', 'Chennai', 'Delhi', '2026-08-15'),  
-> ('F102', 'Mumbai', 'Bangalore', '2026-08-16'),  
-> ('F103', 'Hyderabad', 'Kolkata', '2026-08-17');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> CREATE TABLE Booking (  
->     FlightNo VARCHAR(10),  
->     PassID INT,  
->     Seat VARCHAR(10),  
->     PRIMARY KEY (FlightNo, PassID),  
->     FOREIGN KEY (FlightNo) REFERENCES Flight(FlightNo),  
->     FOREIGN KEY (PassID) REFERENCES Passenger(PassID)  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> INSERT INTO Booking
-> VALUES
-> ('F101', 1, '12A'),
-> ('F101', 2, '12B'),
-> ('F102', 1, '15A'),
-> ('F103', 3, '10C');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Passenger;
```

PassID	PassName
1	Rahul
2	Priya
3	Arun

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Flight;
```

FlightNo	DeptCity	ArrCity	FlightDate
F101	Chennai	Delhi	2026-08-15
F102	Mumbai	Bangalore	2026-08-16
F103	Hyderabad	Kolkata	2026-08-17

```
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Booking;
```

FlightNo	PassID	Seat
F101	1	12A
F101	2	12B
F102	1	15A
F103	3	10C

```
4 rows in set (0.00 sec)
```



```
mysql> SELECT
->     B.FlightNo,
->     P.PassID,
->     P.PassName,
->     B.Seat,
->     F.DeptCity,
->     F.ArrCity,
->     F.FlightDate
-> FROM Booking B
-> JOIN Passenger P
->     ON B.PassID = P.PassID
-> JOIN Flight F
->     ON B.FlightNo = F.FlightNo;
+-----+-----+-----+-----+-----+-----+-----+
| FlightNo | PassID | PassName | Seat | DeptCity | ArrCity | FlightDate |
+-----+-----+-----+-----+-----+-----+-----+
| F101     | 1      | Rahul    | 12A  | Chennai  | Delhi   | 2026-08-15 |
| F102     | 1      | Rahul    | 15A  | Mumbai   | Bangalore | 2026-08-16 |
| F101     | 2      | Priya    | 12B  | Chennai  | Delhi   | 2026-08-15 |
| F103     | 3      | Arun     | 10C  | Hyderabad | Kolkata | 2026-08-17 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Justification

In each decomposed relation, the determinant is a candidate key: Pass ID determines passenger details and Flight No determines flight details. The composite key (Flight No, Pass ID) determines the seat in Booking, so there are no non-key determinants violating BCNF.

5) Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize it to 3NF.

Consider the HR relation:

EMPLOYEE (Emp ID, Emp Name, Dept ID, Dept Name, Manager ID, Manager Name)

Functional Dependencies

Emp ID $\rightarrow$ Emp Name, Dept ID, Manager ID

Dept ID $\rightarrow$ Dept Name

Manager ID $\rightarrow$ Manager Name

Therefore, the transitive dependencies are:

Emp ID $\rightarrow$ Dept ID $\rightarrow$ Dept Name

And

Emp ID → Manager ID → Manager Name
Candidate Key

Emp ID

3NF DECOMPOSITION

EMPLOYEE

EMPLOYEE (Emp ID, Emp Name, Dept ID, Manager ID)

DEPARTMENT

DEPARTMENT (Dept ID, Dept Name)

MANAGER

MANAGER (Manager ID, Manager Name)

This removes the transitive dependencies.

CREATE TABLE AND INSERT DATA

```
mysql> CREATE TABLE Department (  
->     DeptID INT PRIMARY KEY,  
->     DeptName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Department  
-> VALUES  
-> (10, 'Finance'),  
-> (20, 'IT'),  
-> (30, 'HR');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> CREATE TABLE Manager (  
->     ManagerID INT PRIMARY KEY,  
->     ManagerName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Manager  
-> VALUES  
-> (201, 'John'),  
-> (202, 'David'),  
-> (203, 'Sarah');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> CREATE TABLE Employee (  
->     EmpID INT PRIMARY KEY,  
->     EmpName VARCHAR(50),  
->     DeptID INT,  
->     ManagerID INT,  
->     FOREIGN KEY (DeptID) REFERENCES Department(DeptID),  
->     FOREIGN KEY (ManagerID) REFERENCES Manager(ManagerID)  
-> );  
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | ManagerID |
+-----+-----+-----+-----+
| 101   | Rahul   | 10     | 201       |
| 102   | Priya   | 20     | 202       |
| 103   | Arun    | 20     | 202       |
| 104   | Sneha   | 30     | 203       |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Employee;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | ManagerID |
+-----+-----+-----+-----+
| 101   | Rahul   | 10     | 201       |
| 102   | Priya   | 20     | 202       |
| 103   | Arun    | 20     | 202       |
| 104   | Sneha   | 30     | 203       |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Department;
+-----+-----+
| DeptID | DeptName |
+-----+-----+
| 10     | Finance  |
| 20     | IT       |
| 30     | HR       |
+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM Manager;
+-----+-----+
| ManagerID | ManagerName |
+-----+-----+
| 201       | John        |
| 202       | David       |
| 203       | Sarah       |
+-----+-----+
```

```
mysql> SELECT
->     E.EmpID,
->     E.EmpName,
->     D.DeptName,
->     M.ManagerName
-> FROM Employee E
-> JOIN Department D
->     ON E.DeptID = D.DeptID
-> JOIN Manager M
->     ON E.ManagerID = M.ManagerID;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptName | ManagerName |
+-----+-----+-----+-----+
| 101   | Rahul   | Finance  | John        |
| 102   | Priya   | IT       | David       |
| 103   | Arun    | IT       | David       |
| 104   | Sneha   | HR       | Sarah       |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Justification

Each non-key attribute in the decomposed relations depends directly on its respective key. This prevents repeated department and manager information and avoids insertion, deletion, and update anomalies.

6) Case: Library (Member ID, Member Name, Book ID, Title, Author, Genre, Borrow Date). Identify MVDs and decompose to 4NF.

Given Relation

LIBRARY (Member ID, Member Name, Book ID, Title, Author, Genre, Borrow Date)

Assume a member can borrow multiple books and the library can have multiple independent authors/genres associated with books.

The important dependency is:

Member ID $\rightarrow$ Member Name

For the 4NF case, assume each member can have multiple books, independently of another multi-valued fact such as multiple genres of interest.

A cleaner practical relation for demonstrating the MVD is:

MEMBER LIBRARY (Member ID, Book ID, Genre)

with:

Member ID $\twoheadrightarrow$ Book ID

Member ID $\twoheadrightarrow$ Genre

These are multi-valued dependencies.

Example of Redundancy

Suppose Member 101 has:

- **Books: B01, B02**
- **Genres: Fiction, Science**

The relation may contain:

Member ID	Book ID	Genre
101	B01	Fiction
101	B01	Science
101	B02	Fiction
101	B02	Science

The combinations create unnecessary redundancy.

4NF DECOMPOSITION

Separate the independent multi-valued facts.

MEMBER_BOOK

MEMBER BOOK (Member ID, Book ID)

MEMBER_GENRE

MEMBER_GENRE (Member ID, Genre)

CREATE TABLE AND INSERT DATA

```
mysql> CREATE TABLE Member_Library (
->     MemberID INT,
->     BookID VARCHAR(10),
->     Genre VARCHAR(30)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Member_Library
-> VALUES
-> (101, 'B01', 'Fiction'),
-> (101, 'B01', 'Science'),
-> (101, 'B02', 'Fiction'),
-> (101, 'B02', 'Science'),
-> (102, 'B03', 'History'),
-> (102, 'B04', 'History');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Member_Library;
```

MemberID	BookID	Genre
101	B01	Fiction
101	B01	Science
101	B02	Fiction
101	B02	Science
102	B03	History
102	B04	History
101	B01	Fiction
101	B01	Science
101	B02	Fiction
101	B02	Science
102	B03	History
102	B04	History

```
12 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE Member_Book (  
->     MemberID INT,  
->     BookID VARCHAR(10),  
->     PRIMARY KEY (MemberID, BookID)  
-> );
```

```
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Member_Book  
-> VALUES  
-> (101, 'B01'),  
-> (101, 'B02'),  
-> (102, 'B03'),  
-> (102, 'B04');
```

```
Query OK, 4 rows affected (0.01 sec)
```

```
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE TABLE Member_Genre (  
->     MemberID INT,  
->     Genre VARCHAR(30),  
->     PRIMARY KEY (MemberID, Genre)  
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql> INSERT INTO Member_Genre  
-> VALUES  
-> (101, 'Fiction'),  
-> (101, 'Science'),  
-> (102, 'History');
```

Query OK, 3 rows affected (0.01 sec)

Records: 3 Duplicates: 0 Warnings: 0

```
mysql> SELECT * FROM Member_Book;
```

MemberID	BookID
101	B01
101	B02
102	B03
102	B04

4 rows in set (0.00 sec)

```
mysql> SELECT * FROM Member_Genre;
```

MemberID	Genre
101	Fiction
101	Science
102	History

3 rows in set (0.00 sec)

```
mysql> SELECT
->     MB.MemberID,
->     MB.BookID,
->     MG.Genre
-> FROM Member_Book MB
-> JOIN Member_Genre MG
-> ON MB.MemberID = MG.MemberID;
```

MemberID	BookID	Genre
101	B01	Fiction
101	B02	Fiction
101	B01	Science
101	B02	Science
102	B03	History
102	B04	History

6 rows in set (0.00 sec)

Justification

4NF requires every non-trivial multi-valued dependency to have a super key as its determinant. By separating the independent multi-valued attributes into two relations, the MVDs are removed and unnecessary combinations are avoided.

7) Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Consider the telecom billing relation:

BILLING (Bill ID, Customer ID, Customer Name, Plan ID, Plan Name, Amount, Bill Date)

Functional Dependencies

Bill ID $\rightarrow$ Customer ID, Plan ID, Amount, Bill Date

Customer ID $\rightarrow$ Customer Name

Plan ID $\rightarrow$ Plan Name

Therefore:

Bill ID $\rightarrow$ Customer ID $\rightarrow$ Customer Name

And

Bill ID $\rightarrow$ Plan ID $\rightarrow$ Plan Name

are transitive dependencies.

Candidate Key

Bill ID uniquely identifies each bill.

Candidate Key=Bill ID

Primary Key

Bill ID

CREATE TABLE AND INSERT DATA

```
Database changed
mysql> CREATE TABLE Billing (
  ->     BillID INT PRIMARY KEY,
  ->     CustomerID INT,
  ->     CustomerName VARCHAR(50),
  ->     PlanID VARCHAR(10),
  ->     PlanName VARCHAR(50),
  ->     Amount DECIMAL(10,2),
  ->     BillDate DATE
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Billing
  -> VALUES
  -> (1001, 101, 'Rahul', 'P01', '5G Plan', 599, '2026-08-01'),
  -> (1002, 102, 'Priya', 'P02', 'Family Plan', 799, '2026-08-02'),
  -> (1003, 101, 'Rahul', 'P01', '5G Plan', 599, '2026-08-03'),
  -> (1004, 103, 'Arun', 'P03', 'Basic Plan', 399, '2026-08-04');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Billing;
```

BillID	CustomerID	CustomerName	PlanID	PlanName	Amount	BillDate
1001	101	Rahul	P01	5G Plan	599.00	2026-08-01
1002	102	Priya	P02	Family Plan	799.00	2026-08-02
1003	101	Rahul	P01	5G Plan	599.00	2026-08-03
1004	103	Arun	P03	Basic Plan	399.00	2026-08-04

```
4 rows in set (0.00 sec)
```

Justification

Bill ID uniquely identifies every billing record, so it is the candidate key and can be selected as the primary key. Customer ID determines customer information and Plan ID determines plan information, creating transitive dependencies from Bill ID. These dependencies should be considered during normalization to remove redundancy.

8) A retail inventory system has Product (PID, P Name, Supplier ID, Supplier Name, Category, Price, Warehouse). Apply complete normalization.

Given Relation

PRODUCT (PID, P Name, Supplier ID, Supplier Name, Category, Price, Ware house)

Assume:

PID → P Name, Supplier ID, Category, Price, Warehouse
and:

Supplier ID → Supplier Name
Therefore:

PID → Supplier ID → Supplier Name

is a transitive dependency.

Candidate Key

PID

NORMALIZATION

Separate supplier information:

PRODUCT

PRODUCT (PID, P Name, Supplier ID, Category, Price, Warehouse)

SUPPLIER

SUPPLIER (Supplier ID, Supplier Name)

Now Supplier ID is the primary key of SUPPLIER and a foreign key in PRODUCT.

This gives a normalized 3NF/BCNF design under the stated dependencies.

CREATE AND INSERT DATA

```
mysql> CREATE TABLE Supplier (  
mysql> SELECT  
  -> P.PID,  
  -> P.PName,  
  -> P.SupplierID,  
  -> S.SupplierName,  
  -> P.Category,  
  -> P.Price,  
  -> P.Warehouse  
  -> FROM Product P  
  -> JOIN Supplier S  
  -> ON P.SupplierID = S.SupplierID;  
+-----+-----+-----+-----+-----+-----+-----+  
| PID | PName | SupplierID | SupplierName | Category | Price | Warehouse |  
+-----+-----+-----+-----+-----+-----+-----+  
| 101 | Laptop | 201 | ABC Suppliers | Electronics | 65000.00 | Chennai |  
| 103 | Keyboard | 201 | ABC Suppliers | Accessories | 1500.00 | Chennai |  
| 102 | Mouse | 202 | Global Traders | Accessories | 800.00 | Bangalore |  
| 104 | Monitor | 203 | Fresh Supplies | Electronics | 12000.00 | Hyderabad |  
+-----+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.00 sec)  
  -> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql> INSERT INTO Product  
  -> VALUES  
  -> (101, 'Laptop', 201, 'Electronics', 65000, 'Chennai'),  
  -> (102, 'Mouse', 202, 'Accessories', 800, 'Bangalore'),  
  -> (103, 'Keyboard', 201, 'Accessories', 1500, 'Chennai'),  
  -> (104, 'Monitor', 203, 'Electronics', 12000, 'Hyderabad');  
Query OK, 4 rows affected (0.01 sec)  
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Supplier;  
+-----+-----+  
| SupplierID | SupplierName |  
+-----+-----+  
| 201 | ABC Suppliers |  
| 202 | Global Traders |  
| 203 | Fresh Supplies |  
+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql> SELECT * FROM Product;  
+-----+-----+-----+-----+-----+-----+  
| PID | PName | SupplierID | Category | Price | Warehouse |  
+-----+-----+-----+-----+-----+-----+  
| 101 | Laptop | 201 | Electronics | 65000.00 | Chennai |  
| 102 | Mouse | 202 | Accessories | 800.00 | Bangalore |  
| 103 | Keyboard | 201 | Accessories | 1500.00 | Chennai |  
| 104 | Monitor | 203 | Electronics | 12000.00 | Hyderabad |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.00 sec)
```

Justification

The original relation stores supplier information repeatedly for every product supplied by the same supplier. Since $\text{Supplier ID} \rightarrow \text{Supplier Name}$, Supplier Name has a transitive dependency through Supplier ID. Separating the Supplier relation removes this redundancy and gives a normalized 3NF/BCNF design under the stated functional dependencies.

9) Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Consider:

SCHOOL (Student ID, Student Name, Class ID, Class Name, Teacher ID, Teacher Name, Subject ID, Subject Name)

Functional Dependencies

Student ID $\rightarrow$ Student Name, Class ID

Class ID $\rightarrow$ Class Name, Teacher ID

Teacher ID $\rightarrow$ Teacher Name

Subject ID $\rightarrow$ Subject Name

Thus, transitive dependencies include:

Student ID $\rightarrow$ Class ID $\rightarrow$ Class Name, Teacher ID $\rightarrow$ Teacher Name

Candidate Key

Assume a student can study multiple subjects:

(Student ID, Subject ID)

BCNF DECOMPOSITION

STUDENT

STUDENT (Student ID, Student Name, Class ID)

CLASS

CLASS (Class ID, Class Name, Teacher ID)

TEACHER

TEACHER (Teacher ID, Teacher Name)

SUBJECT

SUBJECT (Subject ID, Subject Name)

ENROLLMENT

ENROLLMENT (Student ID, Subject ID)

Each determinant is a key in its respective relation, satisfying BCNF.

CREATE TABLE AND INSERT DATA

```
mysql> CREATE TABLE Teacher (  
->     TeacherID INT PRIMARY KEY,  
->     TeacherName VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Teacher VALUES  
-> (201, 'Kumar'),  
-> (202, 'Priya'),  
-> (203, 'Ravi');  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0  
  
mysql> CREATE TABLE Class (  
->     ClassID INT PRIMARY KEY,  
->     ClassName VARCHAR(50),  
->     TeacherID INT,  
->     FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> INSERT INTO Class VALUES  
-> (1, 'CSE-A', 201),  
-> (2, 'CSE-B', 202),  
-> (3, 'ECE-A', 203);  
Query OK, 3 rows affected (0.01 sec)  
Records: 3  Duplicates: 0  Warnings: 0
```

```

mysql> CREATE TABLE Student (
  ->     StudentID INT PRIMARY KEY,
  ->     StudentName VARCHAR(50),
  ->     ClassID INT,
  ->     FOREIGN KEY (ClassID) REFERENCES Class(ClassID)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student VALUES
  -> (101, 'Rahul', 1),
  -> (102, 'Priya', 2),
  -> (103, 'Arun', 1);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE Subject (
  ->     SubjectID INT PRIMARY KEY,
  ->     SubjectName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO Subject VALUES
  -> (301, 'DBMS'),
  -> (302, 'Python'),
  -> (303, 'Java');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE Enrollment (
  ->     StudentID INT,
  ->     SubjectID INT,
  ->     PRIMARY KEY (StudentID, SubjectID),
  ->     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
  ->     FOREIGN KEY (SubjectID) REFERENCES Subject(SubjectID)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Enrollment VALUES
  -> (101, 301),
  -> (101, 302),
  -> (102, 301),
  -> (103, 303);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

```

```

mysql> SELECT
  ->     S.StudentID,
  ->     S.StudentName,
  ->     C.ClassName,
  ->     T.TeacherName,
  ->     SU.SubjectName
  -> FROM Student S
  -> JOIN Class C ON S.ClassID = C.ClassID
  -> JOIN Teacher T ON C.TeacherID = T.TeacherID
  -> JOIN Enrollment E ON S.StudentID = E.StudentID
  -> JOIN Subject SU ON E.SubjectID = SU.SubjectID;

```

StudentID	StudentName	ClassName	TeacherName	SubjectName
101	Rahul	CSE-A	Kumar	DBMS
101	Rahul	CSE-A	Kumar	Python
102	Priya	CSE-B	Priya	DBMS
103	Arun	CSE-A	Kumar	Java

4 rows in set (0.01 sec)

Justification

The original School ERP relation contains repeated student, class, teacher, and subject information. Decomposing it into separate entities removes the transitive dependencies and stores each fact only once. The resulting relations have determinants that are keys, giving a BCNF design.

10) Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Consider:

STREAMING (Movie ID, Actor ID, Platform ID)

Assume a movie can independently have:

- multiple actors
- multiple streaming platforms

For a 5NF example, assume the valid relationship is represented through three independent pairwise relationships.

Join Dependency

(Movie, Actor), (Movie, Platform), (Actor, Platform)

So decompose into:

MOVIE_ACTOR (Movie ID, Actor ID)

MOVIE_PLATFORM (Movie ID, Platform ID)

ACTOR_PLATFORM (Actor ID, Platform ID)

CREATE TABLES AND INSERT DATA

```
Database changed
mysql> CREATE TABLE Movie_Actor (
    ->     MovieID INT,
    ->     ActorID INT,
    ->     PRIMARY KEY (MovieID, ActorID)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Movie_Actor VALUES
    -> (1, 101),
    -> (1, 102),
    -> (2, 101);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE Movie_Platform (
    ->     MovieID INT,
    ->     PlatformID INT,
    ->     PRIMARY KEY (MovieID, PlatformID)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Movie_Platform VALUES
    -> (1, 201),
    -> (1, 202),
    -> (2, 201);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> CREATE TABLE Actor_Platform (
    ->     ActorID INT,
    ->     PlatformID INT,
    ->     PRIMARY KEY (ActorID, PlatformID)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Actor_Platform VALUES
    -> (101, 201),
    -> (101, 202),
    -> (102, 201),
    -> (102, 202);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```



```
mysql> SELECT
->     MA.MovieID,
->     MA.ActorID,
->     MP.PlatformID
-> FROM Movie_Actor MA
-> JOIN Movie_Platform MP
->     ON MA.MovieID = MP.MovieID
-> JOIN Actor_Platform AP
->     ON MA.ActorID = AP.ActorID
->     AND MP.PlatformID = AP.PlatformID;
```

MovieID	ActorID	PlatformID
1	101	201
1	101	202
1	102	201
1	102	202
2	101	201

5 rows in set (0.00 sec)

Justification

The original movie-streaming relation can contain redundancy when movie, actor, and platform relationships are stored together. Decomposing it into pairwise relations removes the join dependency redundancy. The successful JOIN reconstructs the required tuples, demonstrating a lossless-join decomposition into 5NF.